

**SENA**

**TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE**

**MATERIA:**

**Actividad de proyecto 4**

**Evidencia:**

**GA4-220501095-AA2-EV03. Bases conceptuales acerca del lenguaje unificado de modelado y patrones de diseño.**

**Gerson Jonnathan López Oviedo – 3186647**

**DOCENTE:**

**Arnaldo Alfonso Montiel Brun**

**BOGOTÁ D.C.**

**10 de noviembre del 2025**

## INTRODUCCIÓN

La presente actividad tiene como objetivo profundizar en las bases conceptuales del Lenguaje Unificado de Modelado (UML) y los patrones de diseño, herramientas fundamentales en el desarrollo de software orientado a objetos. A través del análisis de conceptos clave como el control de acceso, la herencia, las asociaciones y los distintos tipos de diagramas UML, se busca fortalecer la capacidad de modelar sistemas de manera estructurada, visual y eficiente.

Este estudio se enmarca dentro del componente formativo del SENA, el cual destaca la importancia del UML como lenguaje gráfico para especificar, construir, visualizar y documentar sistemas. Además, se complementa con ejemplos prácticos y reflexiones técnicas que permiten aplicar estos conceptos en contextos reales, como el diseño de soluciones informáticas y la gestión de requisitos en proyectos de software.

La actividad no solo responde a los requerimientos académicos, sino que también promueve el desarrollo de competencias profesionales en análisis, diseño y documentación de sistemas, preparando al aprendiz para enfrentar desafíos técnicos con claridad, precisión y visión estratégica.

## 1. ¿QUÉ ES UML?

- UML (Unified Modeling Language) es un lenguaje gráfico de modelado que permite especificar, construir, visualizar y documentar sistemas orientados a objetos.

- Surgió como solución a la fragmentación de métodos en la programación orientada a objetos (POO), unificando los enfoques de Booch, Rumbaugh y Jacobson.

### Funciones clave según Booch (2006):

- Visualizar: Representa el sistema de forma comprensible.
- Especificar: Detalla necesidades antes de codificar.
- Construir: Sirve como base para la implementación.
- Documentar: Deja trazabilidad para mantenimiento y evolución.

UML es como el plano arquitectónico de una misión: antes de ejecutar, necesitas visualizar cómo será, qué unidades intervendrán, cómo se comunican y qué objetivos cumplen.

## 2. CONTROL DE ACCESO (PÚBLICO – PRIVADO – PROTEGIDO)

### Desde el PDF (sección Diagramas de clase):

- La **visibilidad** define qué elementos de una clase son accesibles desde otras clases.

| Visibilidad | Símbolo | Acceso desde otras clases                |
|-------------|---------|------------------------------------------|
| Público     | +       | Accesible desde cualquier clase          |
| Privado     | -       | Solo accesible dentro de la misma clase  |
| Protegido   | #       | Accesible desde la clase y sus subclases |

### Ejemplo UML:

+ calcularTotal() // método público

- **precio: double // atributo privado**

**# descuento: float // atributo protegido**

Es como en una base militar: el acceso público es libre, el privado está restringido al comandante, y el protegido solo lo comparten los oficiales de alto rango (subclases).

### 3. HERENCIA

**Desde el PDF (sección 5.2):**

- La **herencia** permite que una clase hija herede atributos y métodos de una clase padre.
- Se representa con una flecha con punta abierta hacia la superclase.

**Visibilidad en herencia:**

- Los elementos públicos y protegidos se heredan.
- Los privados no se heredan directamente.

**Ejemplo gráfico:**

**Clase: Vehículo**



**Clase: Motocicleta**

### 4. ASOCIACIONES

- Las asociaciones representan relaciones entre clases.
- Pueden tener nombre, roles, multiplicidad y navegabilidad.

**Tipos de relaciones:**

- Asociación: Relación simple entre clases.
- Agregación: Relación débil (parte puede existir sin el todo).
- Composición: Relación fuerte (parte depende del todo).

## Ejemplo UML:

Instructor 1 ----- \* Aprendiz

### 5. TIPOS DE DIAGRAMAS UML

Desde el PDF (secciones 1, 2, 4, 5): UML ofrece múltiples diagramas para representar distintos aspectos del sistema.

| Tabla definitiva: Tipos de diagramas UML |                                                                                                                                                  |                                                                             |                                                                                                                                   |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Tipo de diagrama                         | Propósito principal                                                                                                                              | Ejemplo del PDF                                                             | Complemento técnico                                                                                                               |
| Casos de uso                             | Representar las interacciones entre usuarios (actores) y el sistema. Permite identificar funcionalidades clave desde la perspectiva del usuario. | Generar factura, Enviar factura, Buscar factura, Enviar por correo          | Se usa en el levantamiento de requisitos. Incluye relaciones como <i>include</i> , <i>extend</i> y <i>generalization</i> .        |
| Clases                                   | Mostrar la estructura estática del sistema: clases, atributos, métodos y relaciones como herencia, asociación, agregación y composición.         | Cliente, Cajero, Producto, Vehículo, Camioneta                              | Clave para el diseño orientado a objetos. Se representan atributos con visibilidad (+, -, #) y métodos.                           |
| Actividades                              | Representar flujos de trabajo, decisiones, concurrencia y ciclos. Útil para modelar procesos internos del sistema.                               | Proceso de venta, Validación de usuario                                     | Incluye nodos de inicio, decisión, actividades, flujos y nodos finales. Ideal para modelar lógica de negocio.                     |
| Secuencia                                | Mostrar cómo los objetos interactúan en el tiempo, orden de mensajes entre ellos.                                                                | Ejemplo: interacción entre Cliente, Cajero y Sistema en el proceso de venta | Se usa para validar la lógica temporal de procesos como login, compra, envío de datos. Complementa los diagramas de casos de uso. |
| Estados                                  | Representar los diferentes estados que puede tener un objeto durante su ciclo de vida, y las transiciones entre ellos.                           | Ejemplo: estado de una factura (generada, enviada, pagada)                  | Útil para modelar objetos como pedidos, usuarios o documentos que cambian de estado. Se usa en sistemas con lógica de transición. |
| Componentes                              | Mostrar la organización física del sistema: módulos, bibliotecas, servicios.                                                                     | Ejemplo: módulo de facturación, módulo de autenticación                     | Se usa en arquitectura de software para visualizar cómo se distribuyen los componentes y cómo se conectan entre sí.               |
| Despliegue                               | Representar la distribución del sistema en hardware: servidores, dispositivos, redes.                                                            | Ejemplo: servidor web, base de datos, cliente móvil                         | Clave para sistemas distribuidos, aplicaciones web, móviles o IoT. Muestra cómo se instala y ejecuta el software.                 |
| Comunicación                             | Enfocado en los enlaces entre objetos más que en el tiempo. Similar al de secuencia.                                                             | Ejemplo: interacción entre módulos del sistema de ventas                    | Útil para visualizar cómo se comunican los objetos en una operación específica. Se enfoca en los canales de comunicación.         |
| Paquetes                                 | Agrupar clases o componentes relacionados. Ayuda a organizar grandes sistemas.                                                                   | Ejemplo: paquete de ventas, paquete de usuarios                             | Se usa para modularizar el sistema, separar capas (presentación, lógica, datos) y facilitar el mantenimiento y la escalabilidad.  |

# SENA

## SENA

### SENA

#### SENA

##### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA

###### SENA